



中国海洋大学
OCEAN UNIVERSITY OF CHINA

第四周实验报告

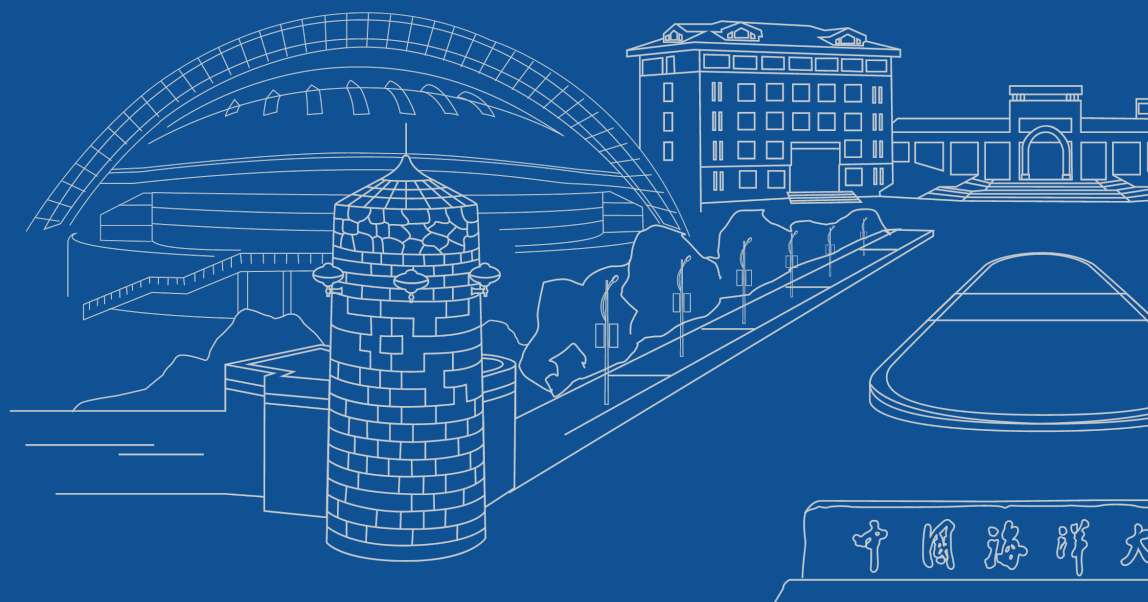
系统开发工具基础

刘晨希

25020007076

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 9 月 14 日



中国海洋大学



第四周实验报告

系统开发工具基础

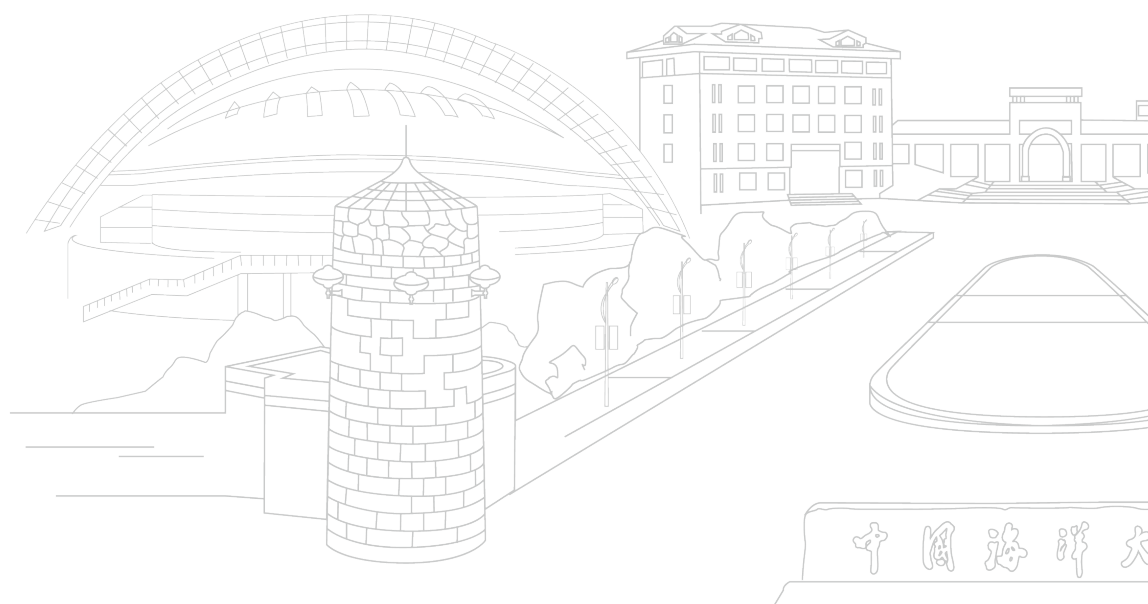
刘晨希

25020007076

指导老师: 周小伟

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 9 月 14 日



目录

目录	i
1 课堂练习及结果	2
1.1 第 13 题建立可执行的本地质量门禁	2
1.1.1 实验内容	2
1.1.2 实验结果	5
1.2 第 14 题让 Make 只重建真正受影响的产物	5
1.2.1 实验内容	5
1.2.2 实验结果	8
1.3 第 15 题把本地 API 数据转换为可读报告	9
1.3.1 实验内容	9
1.3.2 实验结果	11
1.4 第 16 题修复并交付一个陌生的小型工具仓库	11
1.4.1 实验内容	11
1.4.2 实验结果	14
1.5 Github 仓库地址	15
2 综合实例练习：质量门禁、构建自动化与数据管道	16
2.1 本地质量门禁：ruff、pytest 与 check.sh	16
题 1：用 vim 编写带断言的 pytest 用例	16
题 2：在 pyproject.toml 中配置 ruff 与 pytest	17
题 3：用 ruff 统一代码风格并做静态检查	18
题 4：用 pytest 运行测试并读懂报告头	19
题 5：用 chmod 赋予执行权限并读取脚本退出码	19
2.2 用 Make 描述构建链路	20
题 6：用 Make 的依赖关系实现增量构建	20
题 7：用 make clean 清理生成物	21
2.3 本地服务与命令行数据加工	21
题 8：后台启动本地 HTTP 服务并用 jobs 查看作业	21
题 9：用 curl 取回数据、用 jq 过滤与排序	22
2.4 打包交付与产物校验	23
题 10：在虚拟环境中构建 wheel 并校验产物摘要	23
2.5 本章小结	24

目录	1
3 解题感悟	25

课堂练习及结果

1.1 第 13 题建立可执行的本地质量门禁

1.1.1 实验内容

在课程仓库的 w4 目录下完成第四周的四个练习。第十三题复用第三周第十题完成的 greetlab 包，把它复制为 q13 并建立一个轻量、可执行的本地质量检查。

如图 1.1 所示，先进入课程仓库并在 w4 目录下用 `cp -R ../w3/q10 q13` 把第十题的整个工程复制为 q13。由于 q10 目录中残留着上次实验生成的虚拟环境与各类缓存，如图 1.2 所示，用 `rm -rf` 删除 q13/test_env、q13/.pytest_cache 以及 src、tests 下的 `__pycache__` 目录，然后把工程改为由 `pyproject.toml` 统一驱动，不再依赖 q10 目录里的就地虚拟环境。

```
liuchenxi@liuchenxideMacBook-Air lcx_course_work % cd w4
liuchenxi@liuchenxideMacBook-Air w4 % cp -R ../w3/q10 q13
liuchenxi@liuchenxideMacBook-Air w4 %
```

图 1.1: 把第三周的 q10 工程复制为 q13

```
liuchenxi@liuchenxideMacBook-Air w4 % rm -rf q13/test_env q13/.pytest_cache q13/src/greetlab/__pycache__ q13/tests/__pyc|
ache__
liuchenxi@liuchenxideMacBook-Air w4 %
```

图 1.2: 删除 q10 遗留的虚拟环境与缓存目录

如图 1.3 所示，用 heredoc 方式重写 q13/pyproject.toml，在原有构建配置之后补充两段最小配置：`[tool.ruff]` 段把 `line-length` 设为 88；`[tool.ruff.lint]` 段的 `select` 取 `["E", "F"]`，只启用 `pycodestyle` 的 E 类规则与 `Pyflakes` 的 F 类规则；`[tool.pytest.ini_options]` 段把 `pythonpath` 设为 `["src"]`、`testpaths` 设为 `["tests"]`，使 `pytest` 不必额外安装到环境中即可导入 `src` 下的 `greetlab` 包。

```

liuchenxi@liuchenxideMacBook-Air w4 % cat > q13/pyproject.toml <<'EOF'
heredoc> [build-system]
heredoc> requires = ["setuptools>=68"]
heredoc> build-backend = "setuptools.build_meta"
heredoc>
heredoc> [project]
heredoc> name = "greetlab-25020007076"
heredoc> version = "0.1.0"
heredoc>
heredoc> [project.scripts]
heredoc> sdt-greet = "greetlab.cli:main"
heredoc>
heredoc> [tool.ruff]
heredoc> line-length = 88
heredoc>
heredoc> [tool.ruff.lint]
heredoc> select = ["E", "F"]
heredoc>
heredoc> [tool.pytest.ini_options]
heredoc> pythonpath = ["src"]
heredoc> testpaths = ["tests"]
[heredoc> EOF
liuchenxi@liuchenxideMacBook-Air w4 %

```

图 1.3: 在 `pyproject.toml` 中补充 `ruff` 与 `pytest` 的最小配置

如图 1.4 所示，用 `heredoc` 重写 `tests/test_cli.py`，补上一个正常姓名测试和一个空白姓名测试：前者用 `monkeypatch.setattr` 把 `sys.argv` 替换为 `["sdt-greet", "--name", "Alice"]`，调用 `main()` 后通过 `capsys` 捕获标准输出，断言其等于 `"Hello, Alice!\n"`；后者把姓名换成三个空格，用 `pytest.raises(SystemExit)` 捕获退出并把返回码断言为 2。两个用例都含有明确的断言，而不是只调用函数。

```

liuchenxi@liuchenxideMacBook-Air w4 % cat > q13/tests/test_cli.py <<'EOF'
heredoc> import pytest
heredoc> from greetlab.cli import main
heredoc>
heredoc>
heredoc> def test_normal_name(capsys, monkeypatch):
heredoc>     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "Alice"])
heredoc>     main()
heredoc>     captured = capsys.readouterr()
heredoc>     assert captured.out == "Hello, Alice!\n"
heredoc>
heredoc> def test_blank_name_exits_with_2(monkeypatch):
heredoc>     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "   "])
heredoc>     with pytest.raises(SystemExit) as exc:
heredoc>         main()
heredoc>     assert exc.value.code == 2
[heredoc> EOF
liuchenxi@liuchenxideMacBook-Air w4 %

```

图 1.4: 补充正常姓名与空白姓名两个测试

如图 1.5 所示，在 `q13` 目录下依次运行 `ruff format .` 与 `ruff check .`。前者输出 `1 file reformatted, 3 files left unchanged`，就地重排了格式不一致的文件；后者输出 `All checks passed!`。整个过程中没有使用任何工具级别的忽略规则（既没有 `# noqa` 也没有 `lint.ignore` 配置），所有问题都在代码层面修复。

```
liuchenxi@liuchenxideMacBook-Air q13 % ruff check .
All checks passed!
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 1.5: ruff format 与 ruff check 全部通过

如图 1.6 所示,运行 pytest,pytest 读取 pyproject.toml 中的 [tool.pytest.ini_options] 配置,收集到 2 个用例并全部通过,结果为 2 passed in 0.00s。

```
liuchenxi@liuchenxideMacBook-Air q13 % pytest
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w4/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.00s =====
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 1.6: pytest 收集并跑通两个用例

如图 1.7 所示,编写 check.sh: 首行为 shebang #!/usr/bin/env bash, 第二行 set -euo pipefail 让脚本在命令返回非零、引用未定义变量或管道中间环节失败时立即退出,随后依次执行 ruff format --check .、ruff check . 和 pytest。这里用 --check 而不是就地改写,保证门禁只做检查、不修改工作区。

```
liuchenxi@liuchenxideMacBook-Air q13 % cat > check.sh <<'EOF'
heredoc> #!/usr/bin/env bash
heredoc> set -euo pipefail
heredoc>
heredoc> ruff format --check .
heredoc> ruff check .
heredoc> pytest
[heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 1.7: 编写 check.sh 依次执行三项检查

用 chmod +x check.sh 赋予执行权限后,如图 1.8 所示运行 ./check.sh, 输出依次为 4 files already formatted、All checks passed! 与 2 passed in 0.00s, 三项检查全部通过;如图 1.9 所示,用 echo \$? 确认脚本的返回码为 0。若任一环节失败, set -e 会让脚本立即以非零码退出,因此这个返回码可以直接作为后续提交或 CI 判断门禁是否通过的依据。

```
liuchenxi@liuchenxideMacBook-Air q13 % ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w4/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.00s =====
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 1.8: 运行 ./check.sh 三项检查全部通过

```
0  
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 1.9: 确认 `check.sh` 返回码为 0

提交前先清掉本次运行产生的 `.pytest_cache`、`__pycache__` 与 `egg-info` 目录, 如图 1.10 所示, `git add .` 后提交 9f21d2d, 共变更 6 个文件、新增 59 行, 包括 `ai_log.md`、带执行权限的 `check.sh` (权限 100755)、`pyproject.toml`、`src/greetlab/__init__.py`、`src/greetlab/cli.py` 与 `tests/test_cli.py`。

```
[liuchenxi@liuchenxideMacBook-Air q13 % git add .  
[liuchenxi@liuchenxideMacBook-Air q13 % git commit -m "完成了 q13"  
[main 9f21d2d] 完成了 q13  
6 files changed, 59 insertions(+)  
create mode 100644 w4/q13/ai_log.md  
create mode 100755 w4/q13/check.sh  
create mode 100644 w4/q13/pyproject.toml  
create mode 100644 w4/q13/src/greetlab/__init__.py  
create mode 100644 w4/q13/src/greetlab/cli.py  
create mode 100644 w4/q13/tests/test_cli.py  
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 1.10: 第十三题的 `git` 提交记录

1.1.2 实验结果

第十三题在 `q13` 中建立了一道可执行的本地质量门禁: `pyproject.toml` 补充了 `ruff` (`line-length 88, select E/F`) 与 `pytest` (`pythonpath src, testpaths tests`) 的最小配置; `tests/test_cli.py` 补齐正常姓名与空白姓名两个用例, 两者都含有有效断言; `ruff format`、`ruff check` 与 `pytest` 全部通过, 且未使用任何全局忽略规则。`check.sh` 以 `--check` 模式依次执行这三项检查, 运行后返回码为 0, 可作为一个命令被后续提交、`Make` 目标或 `CI` 直接复用。

1.2 第 14 题让 Make 只重建真正受影响的产物

1.2.1 实验内容

第十四题在 `q14` 中按给定内容创建三个源文件, 只需完成 `Makefile`。

如图 1.11 所示, 用 `heredoc` 创建 `data.csv`, 内容为表头 `name,value` 与 `alpha,2,beta,3,gamma,5` 三行数据; 如图 1.12 所示创建 `stats.py`, 用 `csv.DictReader` 逐行读取 `value` 字段求和, 把总数写入 `stats.txt`; 如图 1.13 所示创建 `build_report.py`, 读取 `report.md` 与 `stats.txt` 拼成 `report.txt`; 如图 1.14 所示创建只有一行标题 `# Data Report` 的模板 `report.md`。四者构成一条数据链路: `data.csv` 经 `stats.py` 汇总出 `stats.txt`, 再由 `build_report.py` 结合模板 `report.md` 生成最终产物 `report.txt`。

```
liuchenxi@liuchenxideMacBook-Air q14 % cat > data.csv <<'EOF'
heredoc> name,value
heredoc> alpha,2
heredoc> beta,3
heredoc> gamma,5
heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.11: 创建数据文件 *data.csv*

```
liuchenxi@liuchenxideMacBook-Air q14 % cat > stats.py <<'EOF'
heredoc> import csv
heredoc> with open("data.csv") as f:
heredoc>     total = sum(int(r["value"]) for r in csv.DictReader(f))
heredoc> open("stats.txt", "w").write(str(total))
heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.12: 创建 *stats.py* 统计 *value* 总和

```
liuchenxi@liuchenxideMacBook-Air q14 % cat > build_report.py <<'EOF'
heredoc> text = open("report.md").read()
heredoc> total = open("stats.txt").read()
heredoc> open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.13: 创建 *build_report.py* 合成最终产物

```
liuchenxi@liuchenxideMacBook-Air q14 % cat > report.md <<'EOF'
heredoc> # Data Report
heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.14: 创建模板文件 *report.md*

如图 1.15 所示, 用 `cat -et Makefile` 查看写好的 *Makefile* (`-e` 把行尾的 `$` 与制表符显示出来, 便于确认配方行确实以 `TAB` 开头)。文件内容为: `.PHONY: all clean` 声明 `all` 与 `clean` 是伪目标, 避免与同名文件冲突; `all: report.txt` 把最终产物 `report.txt` 作为默认目标; `stats.txt: data.csv stats.py` 声明中间产物依赖数据文件与脚本, 配方为 `python3 stats.py`; `report.txt: report.md stats.txt build_report.py` 声明最终产物依赖模板、中间产物与脚本, 配方为 `python3 build_report.py`; `clean:` 的配方为 `rm -f stats.txt report.txt`。整条链路的依赖被完整描述出来, `make` 才能据此判断哪一步真正需要重跑。

```
[liuchenxi@liuchenxideMacBook-Air q14 % cat -et Makefile
.PHONY: all clean$
$
all: report.txt$
$
stats.txt: data.csv stats.py$
^Ipython3 stats.py$
$
report.txt: report.md stats.txt build_report.py$
^Ipython3 build_report.py$
$
clean:$
^Irm -f stats.txt report.txt$
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.15: 查看 Makefile 的依赖与配方

如图 1.16 所示,首次运行 make 时两个产物都不存在,make 依次执行 python3 stats.py 与 python3 build_report.py; 用 ls 确认目录中出现了 stats.txt 与 report.txt, 如图 1.17 所示, cat stats.txt 得到 10 (2+3+5), cat report.txt 得到标题 # Data Report、一个空行与 Total: 10。

```
[liuchenxi@liuchenxideMacBook-Air q14 % make
python3 stats.py
python3 build_report.py
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.16: 首次 make 依次生成两个产物

```
[liuchenxi@liuchenxideMacBook-Air q14 % cat stats.txt
10%
liuchenxi@liuchenxideMacBook-Air q14 % cat report.txt
# Data Report

Total: 10
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.17: 查看 stats.txt 与 report.txt 的内容

如图 1.18 所示,不修改任何文件再次运行 make, 输出 make: Nothing to be done for `all'。因为 stats.txt 比 data.csv 与 stats.py 新, report.txt 又比它的三个依赖新, 所有目标都是最新的, make 没有执行任何配方。

```
[liuchenxi@liuchenxideMacBook-Air q14 % make
make: Nothing to be done for `all'.
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.18: 无改动时第二次 make 不执行任何配方

如图 1.19 所示, 用 `touch data.csv` 只更新数据文件的时间戳 (内容不变), 随后再次运行 `make`。make 发现 `data.csv` 比 `stats.txt` 新, 于是重跑 `stats.py` 更新 `stats.txt`; 又因为 `report.txt` 依赖 `stats.txt`, 随之重跑 `build_report.py`。这一步验证了依赖链是级联生效的: 只改动最上游的输入, 受影响的中间产物与最终产物都会按序重建。

```
[liuchenxi@liuchenxideMacBook-Air q14 % touch data.csv
[liuchenxi@liuchenxideMacBook-Air q14 % make
python3 stats.py
python3 build_report.py
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.19: `touch data.csv` 后 `make` 级联重建两个产物

如图 1.20 所示, 运行 `make clean`, 执行配方 `rm -f stats.txt report.txt`, 只删除两个生成物; 如图 1.21 所示, 用 `ls` 确认目录中只剩下 `build_report.py`、`data.csv`、`Makefile`、`report.md`、`stats.py` 五个源文件与构建文件, 中间产物与最终产物都被清理干净。

```
[liuchenxi@liuchenxideMacBook-Air q14 % make clean
rm -f stats.txt report.txt
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.20: `make clean` 只删除生成物

```
[liuchenxi@liuchenxideMacBook-Air q14 % ls
build_report.py data.csv      Makefile      report.md     stats.py
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.21: `clean` 之后目录中只剩源文件

如图 1.22 所示, `git add .` 后提交 991bebf, 变更 5 个文件、新增 24 行, 即 `Makefile`、`build_report.py`、`data.csv`、`report.md` 与 `stats.py`。

```
[liuchenxi@liuchenxideMacBook-Air q14 % git add .
[liuchenxi@liuchenxideMacBook-Air q14 % git commit -m "完成了 q14"
[main 991bebf] 完成了 q14
5 files changed, 24 insertions(+)
create mode 100644 w4/q14/Makefile
create mode 100644 w4/q14/build_report.py
create mode 100644 w4/q14/data.csv
create mode 100644 w4/q14/report.md
create mode 100644 w4/q14/stats.py
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 1.22: 第十四题的 `git` 提交记录

1.2.2 实验结果

第十四题的 `Makefile` 用「目标: 依赖 + 配方」的方式完整描述了 `data.csv` → `stats.txt` → `report.txt` 的构建链路, 并把 `all` 与 `clean` 声明为伪目标。首次 `make` 生成 `stats.txt` 与 `report.txt` 两个产物 (分别为 10 与「# Data Report / Total: 10」); 无改动时第二次 `make` 提示

Nothing to be done, 说明增量判断生效; touch data.csv 后 make 级联重建了 stats.txt 与 report.txt; make clean 只删除这两个生成物, 五个源文件全部保留。

1.3 第 15 题把本地 API 数据转换为可读报告

1.3.1 实验内容

如图 1.23 所示, 在 q15 中用 heredoc 把给定 JSON 保存为 packages.json, 共 5 条记录, 每条包含 name、status、downloads、version 四个字段; 用 jq 'length' packages.json 确认文件可被正确解析并返回 5。

```
liuchenxi@liuchenxideMacBook-Air q15 % cat > packages.json <<'EOF'
heredoc> [
heredoc>   {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
heredoc>   {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
heredoc>   {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
heredoc>   {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
heredoc>   {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
heredoc> ]
[heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q15 %
```

图 1.23: 保存给定的 packages.json

如图 1.24 所示, 用 python3 -m http.server 8000 > server.log 2>&1 & 在 q15 目录后台启动本地 HTTP 服务, 把标准输出与标准错误一起重定向到 server.log, 作业号为 36391; 再用 jobs 确认该任务处于 running 状态。

```
5
liuchenxi@liuchenxideMacBook-Air q15 % python3 -m http.server 8000 > server.log 2>&1 &
[1] 36391
liuchenxi@liuchenxideMacBook-Air q15 %
```

图 1.24: 后台启动本地 HTTP 服务并确认作业状态

如图 1.25 所示, 用 curl -fsS http://127.0.0.1:8000/packages.json 从本地服务取回数据。其中 -f 让服务器返回 4xx/5xx 时以非零码退出, 而不是把错误页当成数据继续处理; -s 关闭进度与统计信息; -S 保证在出错时仍然打印错误原因, 因此这条命令既安静又不会「静默失败」。终端中回显出的正是 packages.json 的原始内容。

```
liuchenxi@liuchenxideMacBook-Air q15 % jobs
[1] + running  python3 -m http.server 8000 > server.log 2>&1
liuchenxi@liuchenxideMacBook-Air q15 % curl -fsS http://127.0.0.1:8000/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

图 1.25: 用 curl -fsS 从本地服务获取 JSON 数据

如图 1.26 所示, 用管道把 curl 的输出交给 jq 依次展开、过滤与排序: 先用 .[] 展开数组, 再用 select(.status == "active" and .downloads >= 100) 保留 status 为 active 且 downloads 不少于 100 的记录, 筛掉 beta (inactive) 与 epsilon (80), 最后用 sort_by(-.downloads, .name) 先按 downloads 降序、再按 name 升序排列。最终输出的

顺序为 delta(450)、gamma(450)、alpha(120)——delta 与 gamma 下载量相同，按字母序 delta 排在前面。

```
[liuchenxi@liuchenxideMacBook-Air q15 % curl -fsS http://127.0.0.1:8000/packages.json | jq '[.] | select(.status == "active" and .downloads >= 100)] | sort_by(-.downloads, .name)']
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
```

图 1.26: 用 jq 过滤 active 且下载量不少于 100 的记录并排序

如图 1.27 所示，用 vim 编写 api_report.sh，把上面的 curl 与 jq 固化成一条可重复执行的命令：jq 使用 -r 输出原始文本，先构造 Markdown 的表头（# Package Report 标题、空行、name/version/downloads 三列表头与右对齐行），再用 map("| \(.name) | \(.version) | \(.downloads) |") 把每条记录格式化表格行，展平后逐行输出，整体重定向到 summary.md。chmod +x 后运行 ./api_report.sh，再用 cat summary.md 查看结果：表格共 3 行，依次为 delta(0.9.0, 450)、gamma(1.5.1, 450)、alpha(1.2.0, 120)，正是被筛选并排序后的记录。

```
[liuchenxi@liuchenxideMacBook-Air q15 % vim api_report.sh
[liuchenxi@liuchenxideMacBook-Air q15 % chmod +x api_report.sh
[liuchenxi@liuchenxideMacBook-Air q15 % ./api_report.sh
[liuchenxi@liuchenxideMacBook-Air q15 % cat summary.md
# Package Report

| name | version | downloads |
| --- | --- | ---: |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 1.27: 编写并运行 api_report.sh 生成 summary.md

如图 1.28 所示，用 ls -l 确认目录下只有 api_report.sh、packages.json、server.log、summary.md 四个文件，其中 api_report.sh 带有执行权限。

```
[liuchenxi@liuchenxideMacBook-Air q15 % ls -l
total 32
-rwxr-xr-x 1 liuchenxi staff 359  9月 14 12:57 api_report.sh
-rw-r--r-- 1 liuchenxi staff 400  9月 14 12:46 packages.json
-rw-r--r-- 1 liuchenxi staff 240  9月 14 12:57 server.log
-rw-r--r-- 1 liuchenxi staff 142  9月 14 12:57 summary.md
```

图 1.28: 查看 q15 目录的内容与文件权限

如图 1.29 所示，git add q15 后提交 1f41e3f，变更 4 个文件、新增 34 行。

```
[liuchenxi@liuchenxideMacBook-Air w4 % git add q15
[liuchenxi@liuchenxideMacBook-Air w4 % git commit -m "完成了q15"
[main 1f41e3f] 完成了q15
 4 files changed, 34 insertions(+)
 create mode 100755 w4/q15/api_report.sh
 create mode 100644 w4/q15/packages.json
 create mode 100644 w4/q15/server.log
 create mode 100644 w4/q15/summary.md
```

图 1.29: 第十五题的 git 提交记录

1.3.2 实验结果

第十五题把本地数据放到了「HTTP 服务 + curl + jq」的链路上: `packages.json` 由 `python -m http.server 8000` 通过 `http://127.0.0.1:8000/packages.json` 提供; `curl` 用 `-fsS` 获取, 保证出错即失败而不静默; `jq` 按 `status == "active"` 且 `downloads >= 100` 过滤后按下载量降序、名称升序排序, 得到 `delta` 与 `gamma` (各 450) 在前、`alpha` (120) 在后, `beta` 与 `epsilon` 被正确筛除。 `api_report.sh` 把这条链路固化为一条命令并生成 `summary.md`, 其标题与 `name/version/downloads` 三列表格均符合要求。

1.4 第 16 题修复并交付一个陌生的小型工具仓库

1.4.1 实验内容

第十六题复用第十三题的项目, 在 `q16` 中完成一次 15 分钟的综合检查。

如图 1.30 所示, 在 `w4` 下用 `cp -R q13 q16` 把 `q13` 整体复制为 `q16`; 进入后 `ls` 可以看到 `ai_log.md`、`check.sh`、`pyproject.toml`、`src`、`tests` 五个条目, 即一个已经配好质量门禁的小型 Python 工程。

```
[liuchenxi@liuchenxideMacBook-Air lcx_course_work % cd w4
[liuchenxi@liuchenxideMacBook-Air w4 % cp -R q13 q16
[liuchenxi@liuchenxideMacBook-Air w4 % cd q16
[liuchenxi@liuchenxideMacBook-Air q16 % ls
ai_log.md      check.sh      pyproject.toml src           tests
```

图 1.30: 把 `q13` 复制为 `q16` 并查看目录结构

如图 1.31 所示, 用 `vim` 把 `src/greetlab/cli.py` 的问候语句子临时改成始终返回字面量的 `print("Hello, name!")` (即去掉 `f-string` 的 `f` 前缀), 人为制造一个「输出与参数无关」的缺陷; `cat` 查看文件确认改动已经生效。

```

[liuchenxi@liuchenxideMacBook-Air q16 % vim src/greetlab/cli.py
[liuchenxi@liuchenxideMacBook-Air q16 % cat src/greetlab/cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()

    if not a.name.strip():
        p.error("--name must not be blank")

    print("Hello, name!")

```

图 1.31: 把问候语临时改为始终返回字面量

如图 1.32 所示, 运行 `./check.sh`: `ruff format` 与 `ruff check` 都通过, 但 `pytest` 失败——`tests/test_cli.py::test_normal_name` 在断言 `captured.out == "Hello, Alice!\n"` 时实际得到 `'Hello, name!\n'`, 抛出 `AssertionError`, 结果为 `1 failed, 1 passed in 0.02s`。值得注意的是空白姓名用例仍然通过, 因为参数校验逻辑没有被改动, 这正是「测试只暴露了部分症状」的典型情形。

```

liuchenxi@liuchenxideMacBook-Air q16 % ./check.sh
4 files already formatted
All checks passed!

===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w4/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py F. [100%]

===== FAILURES =====
_____ test_normal_name _____

capsys = <_pytest.capture.CaptureFixture object at 0x108641ba0>
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x10863d480>

    def test_normal_name(capsys, monkeypatch):
        monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "Alice"])
        main()
        captured = capsys.readouterr()
        assert captured.out == "Hello, Alice!\n"
E       AssertionError: assert 'Hello, name!\n' == 'Hello, Alice!\n'
E
E       - Hello, Alice!
E       + Hello, name!

tests/test_cli.py:9: AssertionError

===== short test summary info =====
FAILED tests/test_cli.py::test_normal_name - AssertionError: assert 'Hello, name!\n' == 'Hello, Alice!\n'
===== 1 failed, 1 passed in 0.02s =====

```

图 1.32: 注入缺陷后 `check.sh` 中 `test_normal_name` 失败

如图 1.33 所示, 把 `cli.py` 恢复成 f-string 形式的 `print(f"Hello, {a.name}!")`, 再次运行 `./check.sh`, 输出 `4 files already formatted`、`All checks passed!` 与 `2 passed in 0.00s`, 两个用例全部通过。

```

[liuchenxi@liuchenxideMacBook-Air q16 % vim src/greetlab/cli.py
[liuchenxi@liuchenxideMacBook-Air q16 % ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w4/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..

===== 2 passed in 0.00s =====

```

图 1.33: 修复问候语后 check.sh 全部通过

如图 1.34 所示, 编写最小 Makefile: `.PHONY: check build clean` 声明三个伪目标; `check:` 的配方为 `./check.sh`, 直接复用第十三题建立的门禁; `build:` 的配方为 `python3 -m build --wheel`; `clean:` 的配方为 `rm -rf build dist *.egg-info`。

```

[liuchenxi@liuchenxideMacBook-Air q16 % vim Makefile
[liuchenxi@liuchenxideMacBook-Air q16 % cat Makefile
.PHONY: check build clean

check:
    ./check.sh

build:
    python3 -m build --wheel

clean:
    rm -rf build dist *.egg-info

```

图 1.34: 编写 q16 的最小 Makefile

如图 1.35 所示, 运行 `make check`, `make` 先回显它执行的命令行 `./check.sh`, 随后脚本的三项检查全部通过、2 passed。

```

[liuchenxi@liuchenxideMacBook-Air q16 % make check
./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w4/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..

===== 2 passed in 0.00s =====

```

图 1.35: make check 复用 check.sh 并全部通过

如图 1.36 所示, 运行 `make build` 时先遇到一次失败: `python3` 报 `No module named build`, `make` 以 `*** [build] Error 1` 结束, `ls dist` 也提示 `dist: No such file or directory`。原因是当前 shell 使用的 Homebrew Python 环境里没有安装 `build` 构建工具。如图 1.37 所示, 在项目虚拟环境 (提示符前缀为 `(.venv)`) 中再次执行 `make build`, 构建过程开始创建隔离环境并安装 `setuptools`; 最终如图 1.38 所示, 输出 `Successfully built greetlab_25020007076-0.1.0-py3-none-any.whl`, 在 `dist` 目录下成功产出 wheel。用 `shasum -a 256` 对生成的 wheel 计算 SHA-256 摘要, 结果为 `c3e33ba5b23525ec6c0e38cbbb48d88c8c99f42fe14b5966b2093177595d8701`。

```
[liuchenxi@liuchenxideMacBook-Air q16 % make build
python3 -m build --wheel
/opt/homebrew/opt/python@3.14/bin/python3.14: No module named build
make: *** [build] Error 1
[liuchenxi@liuchenxideMacBook-Air q16 % ls dist
ls: dist: No such file or directory
```

图 1.36: 缺少 *build* 工具时 *make build* 失败

```
[(.venv) liuchenxi@liuchenxideMacBook-Air q16 % make build
python3 -m build --wheel
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for wheel...
```

图 1.37: 在虚拟环境中重新执行 *make build*

```
adding 'greetlab_25020007076-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'
removing build/bdist.macosx-26.0-arm64/wheel
Successfully built greetlab_25020007076-0.1.0-py3-none-any.whl
```

图 1.38: 成功构建出 *greetlab* 的 *wheel* 产物

如图 1.39 所示, `git add .` 后提交 `3fffeeaa`, 共变更 18 个文件、新增 97 行, 包括 `Makefile`、`check.sh`、修复后的 `src/greetlab/cli.py`、`tests/test_cli.py`、`ai_log.md` 以及 `dist` 下生成的 *wheel*。

```
[(.venv) liuchenxi@liuchenxideMacBook-Air q16 % git add .
[liuchenxi@liuchenxideMacBook-Air q16 % git commit -m "完成了q16"
[main 3fffeeaa] 完成了q16
18 files changed, 97 insertions(+)
create mode 100644 w4/q16/Makefile
create mode 100644 w4/q16/ai_log.md
create mode 100644 w4/q16/build/lib/greetlab/__init__.py
create mode 100644 w4/q16/build/lib/greetlab/cli.py
create mode 100755 w4/q16/check.sh
create mode 100644 w4/q16/dist/greetlab_25020007076-0.1.0-py3-none-any.whl
create mode 100644 w4/q16/pyproject.toml
create mode 100644 w4/q16/src/greetlab/__init__.py
create mode 100644 w4/q16/src/greetlab/__pycache__/__init__.cpython-314.pyc
create mode 100644 w4/q16/src/greetlab/__pycache__/cli.cpython-314.pyc
create mode 100644 w4/q16/src/greetlab/cli.py
create mode 100644 w4/q16/src/greetlab_25020007076.egg-info/PKG-INFO
create mode 100644 w4/q16/src/greetlab_25020007076.egg-info/SOURCES.txt
create mode 100644 w4/q16/src/greetlab_25020007076.egg-info/dependency_links.txt
create mode 100644 w4/q16/src/greetlab_25020007076.egg-info/entry_points.txt
create mode 100644 w4/q16/src/greetlab_25020007076.egg-info/top_level.txt
create mode 100644 w4/q16/tests/__pycache__/test_cli.cpython-314-pytest-9.1.1.pyc
create mode 100644 w4/q16/tests/test_cli.py
```

图 1.39: 第十六题的 *git* 提交记录

1.4.2 实验结果

第十六题在复用的 `q13` 工程上完成了一次综合检查: 注入「问候语写死为字面量」的缺陷后, `check.sh` 中只有 `test_normal_name` 失败 (1 failed, 1 passed), 把实现改回 `f-string` 后三项检查全部通过。最小 `Makefile` 提供 `check`、`build`、`clean` 三个目标, `make check`

复用 `check.sh` 并全部通过；`make build` 在缺少 `build` 工具的环境下会以 `Error 1` 明确失败，在装有该工具的虚拟环境中则成功产出 `greetlab_25020007076-0.1.0-py3-none-any.whl`，其 SHA-256 为 `c3e33ba5b2...595d8701`。最终改动以一次提交 `3fffee`（18 个文件、97 行新增）交付。

1.5 Github 仓库地址

本次课程的报告、练习、代码等，均已上传至个人 `github` 仓库。
Github 仓库地址：https://github.com/lcx821/lcx_course_work

综合实例练习：质量门禁、构建自动化与数据管道

本章把第四周课上讲到的操作拆成 10 个最小实例，每个实例只做一件事：要么是一条命令，要么是一次编辑，当场执行、当场看输出。四个小节分别对应本周的四道练习题——本地质量门禁（第 13 题）、用 Make 描述构建链路（第 14 题）、本地服务与命令行数据处理（第 15 题）、打包交付与产物校验（第 16 题）。所有命令都在课程仓库的 w4 目录下执行，截图取自实际操作过程。

2.1 本地质量门禁：ruff、pytest 与 check.sh

题 1：用 vim 编写带断言的 pytest 用例

要求：质量门禁的第一步是「有测试可跑」。用 vim 在 tests/test_cli.py 中补上两个用例：一个正常姓名、一个只含空白字符的姓名。两个用例都必须有明确断言，而不是只把被测函数调用一遍。

操作命令：

```
cd w4/q13
vim tests/test_cli.py
cat tests/test_cli.py
```

用例代码：

```
def test_normal_name(capsys, monkeypatch):
    monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "Alice"])
    main()
    captured = capsys.readouterr()
    assert captured.out == "Hello, Alice!\n"

def test_blank_name_exits_with_2(monkeypatch):
    monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])
    with pytest.raises(SystemExit) as exc:
        main()
    assert exc.value.code == 2
```

结果说明：如图 2.1 所示，*vim* 编辑结束后用 *cat* 回显整个文件，可以看到两个用例把断言落在了可观察的行为上：前者用 *monkeypatch.setattr* 把 *sys.argv* 换成 `["sdt-greet", "--name", "Alice"]`，调用 *main()* 后用 *capsys* 捕获标准输出，断言其等于 `"Hello, Alice!\n"`；后者把姓名换成三个空格，用 *pytest.raises(SystemExit)* 捕获退出并把返回码断言为 2。这样写的用例在实现被改坏时会真的失败，而不是永远通过。

```
liuchenxi@liuchenxideMacBook-Air w4 % cat > q13/tests/test_cli.py <<'EOF'
heredoc> import pytest
heredoc> from greetlab.cli import main
heredoc>
heredoc>
heredoc> def test_normal_name(capsys, monkeypatch):
heredoc>     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "Alice"])
heredoc>     main()
heredoc>     captured = capsys.readouterr()
heredoc>     assert captured.out == "Hello, Alice!\n"
heredoc>
heredoc> def test_blank_name_exits_with_2(monkeypatch):
heredoc>     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "  "])
heredoc>     with pytest.raises(SystemExit) as exc:
heredoc>         main()
heredoc>     assert exc.value.code == 2
[heredoc> EOF
liuchenxi@liuchenxideMacBook-Air w4 %
```

图 2.1: 用 *vim* 编写两个带断言的测试用例并 *cat* 回显

题 2：在 *pyproject.toml* 中配置 *ruff* 与 *pytest*

要求：让工具从项目清单里读配置，而不是每次在命令行上敲参数。用 *vim* 在 *q13/pyproject.toml* 末尾追加三段最小配置，再用 *cat* 回显确认。

操作命令：

```
cd w4
vim q13/pyproject.toml
cat q13/pyproject.toml
```

追加的配置：

```
[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

结果说明：如图 2.2 所示，回显出的 *pyproject.toml* 除了原有的 `[build-system]`、`[project]`、`[project.scripts]` 之外，末尾多了三段：`[tool.ruff]` 把行宽定为 88；`[tool.ruff.lint]` 的 `select` 只启用 *pycodestyle* 的 E 类与 *Pyflakes* 的 F 类规则；

[tool.pytest.ini_options] 把 pythonpath 设为 ["src"]、testpaths 设为 ["tests"], 于是 pytest 不用被安装进环境也能导入 src 下的 greetlab 包。

```
[liuchenxi@liuchenxideMacBook-Air w4 % cat q13/pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007076"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
liuchenxi@liuchenxideMacBook-Air w4 %
```

图 2.2: pyproject.toml 中的 ruff 与 pytest 配置

题 3：用 ruff 统一代码风格并做静态检查

要求：一条命令统一格式、一条命令做静态检查，两条命令都不修改检查规则。

操作命令：

```
cd q13
ruff format .
ruff check .
```

结果说明：如图 2.3 所示，ruff format . 就地重排格式不一致的文件，输出 1 file reformatted, 3 files left unchanged，即扫描到的 4 个 Python 文件中只有 1 个需要改写；如图 2.4 所示，ruff check . 输出 All checks passed!，E、F 两类规则全部通过。

```
[liuchenxi@liuchenxideMacBook-Air w4 % cd q13
[liuchenxi@liuchenxideMacBook-Air q13 % ruff format .
1 file reformatted, 3 files left unchanged
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 2.3: ruff format 就地统一格式

```
liuchenxi@liuchenxideMacBook-Air q13 % ruff check .  
All checks passed!  
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 2.4: ruff check 静态检查全部通过

题 4：用 pytest 运行测试并读懂报告头

要求：直接运行 pytest，并观察报告里的 rootdir、configfile、testpaths 三个字段分别来自哪里。

操作命令：

```
cd q13  
pytest
```

结果说明：如图 2.5 所示，报告头给出 platform darwin —— Python 3.14.7、pytest-9.1.1；configfile: pyproject.toml 说明配置来自上一个实例写进项目清单的内容；testpaths: tests 说明收集范围被限定在 tests 目录，因此不需要在命令行上再传路径；随后 collected 2 items 收集到 2 个用例，tests/test_cli.py .. 的两个点表示两个用例都通过，最后一行 2 passed in 0.00s 是本次运行的结果。

```
liuchenxi@liuchenxideMacBook-Air q13 % pytest  
===== test session starts =====  
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0  
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w4/q13  
configfile: pyproject.toml  
testpaths: tests  
collected 2 items  
  
tests/test_cli.py .. [100%]  
  
===== 2 passed in 0.00s =====  
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 2.5: pytest 读取 pyproject.toml 配置并跑通两个用例

题 5：用 chmod 赋予执行权限并读取脚本退出码

要求：把三条检查固化成 check.sh 之后，给它执行权限，运行脚本，并用 echo \$? 读取返回码。

操作命令：

```
chmod +x check.sh  
./check.sh  
echo $?
```

结果说明：如图 2.6 所示，chmod +x check.sh 执行后没有任何输出、直接回到提示符，说明权限位已经加上；脚本首行的 set -euo pipefail 保证任一环节失败就立即以非零码退出。如图 2.7 所示，./check.sh 依次输出 4 files already formatted、All checks passed! 与 2 passed in 0.00s，三项检查全部通过，紧随其后的 echo \$? 打印 0，这个返回码可以直接被后续命令用来判断门禁是否通过。

```
[heredoc> EOF
liuchenxi@liuchenxideMacBook-Air q13 % chmod +x check.sh
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 2.6: `chmod +x` 给 `check.sh` 加上执行权限

```
0
liuchenxi@liuchenxideMacBook-Air q13 %
```

图 2.7: 用 `echo $?` 确认 `check.sh` 的返回码为 0

2.2 用 Make 描述构建链路

题 6：用 Make 的依赖关系实现增量构建

要求：Makefile 把 `data.csv` → `stats.txt` → `report.txt` 的依赖写清楚之后，观察首次构建、无改动构建、只改时间戳三种情况分别发生什么。

操作命令：

```
cd w4/q14
make          # 首次：两个产物都不存在
make          # 无改动
touch data.csv # 只更新最上游输入的时间戳，内容不变
make          # 观察级联重建
```

结果说明：如图 2.8 所示，第二次不带任何改动的 `make` 直接提示 `make: Nothing to be done for `all'.`，因为 `stats.txt` 比 `data.csv` 与 `stats.py` 新、`report.txt` 又比它的三个依赖新，所有目标都是最新的，一条配方都没有执行；随后 `touch data.csv` 只更新数据文件的时间戳。如图 2.9 所示，再次 `make` 时 `make` 发现 `data.csv` 比 `stats.txt` 新，于是重跑 `python3 stats.py` 更新中间产物，又因为 `report.txt` 依赖 `stats.txt`，顺势重跑 `python3 build_report.py` 更新最终产物——只改动最上游的输入，受影响的产物就按依赖顺序级联重建。

```
make: Nothing to be done for `all'.
liuchenxi@liuchenxideMacBook-Air q14 % touch data.csv
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 2.8: 无改动时 `make` 不做事，`touch` 之后才动最上游输入

```
liuchenxi@liuchenxideMacBook-Air q14 % touch data.csv
liuchenxi@liuchenxideMacBook-Air q14 % make
python3 stats.py
python3 build_report.py
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 2.9: `touch data.csv` 后 `make` 级联重建两个产物

题 7：用 make clean 清理生成物

要求：把中间产物与最终产物删掉，只保留源文件与构建文件。

操作命令：

```
make clean  
ls
```

结果说明：如图 2.10 所示，make clean 先回显它执行的命令行 `rm -f stats.txt report.txt`，说明 clean 的配方只删这两个生成物，不会碰到源文件；如图 2.11 所示，ls 确认目录中只剩下 Makefile、build_report.py、data.csv、report.md、stats.py 五个源文件与构建文件，中间产物与最终产物都被清理干净。

```
[liuchenxi@liuchenxideMacBook-Air q14 % make clean  
rm -f stats.txt report.txt  
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 2.10: make clean 执行 rm -f 只删除生成物

```
[liuchenxi@liuchenxideMacBook-Air q14 % ls  
build_report.py data.csv      Makefile      report.md     stats.py  
liuchenxi@liuchenxideMacBook-Air q14 %
```

图 2.11: clean 之后目录中只剩源文件

2.3 本地服务与命令行数据加工

题 8：后台启动本地 HTTP 服务并用 jobs 查看作业

要求：把当前目录变成一个本地 HTTP 服务，把这条命令放到后台运行，重定向日志，并用 jobs 确认作业状态。

操作命令：

```
cd w4/q15  
python3 -m http.server 8000 > server.log 2>&1 &  
jobs
```

结果说明：如图 2.12 所示，命令末尾的 & 让服务在后台运行，> server.log 2>&1 把标准输出与标准错误一并重定向到 server.log，shell 随即打印作业号 36391；接着 jobs 列出该作业仍处于运行状态，说明服务已在 8000 端口监听，packages.json 可以经由 http://127.0.0.1:8000/packages.json 取到。要看服务运行期间的请求记录，直接查看 server.log 即可；服务用完后可用 kill %1 结束这个后台作业。

```

5
[liuchenxi@liuchenxideMacBook-Air q15 % python3 -m http.server 8000 > server.log 2>&1 &
[1] 36391
liuchenxi@liuchenxideMacBook-Air q15 % █

```

图 2.12: 后台启动本地 HTTP 服务并用 jobs 确认作业状态

题 9：用 curl 取回数据、用 jq 过滤与排序

要求：从本地服务取回 JSON，先用 jq 统计记录条数，再按「status 为 active 且 downloads 不少于 100」过滤，并按下载量降序、名称升序排序。

操作命令：

```

curl -fsS http://127.0.0.1:8000/packages.json
curl -fsS http://127.0.0.1:8000/packages.json | jq 'length'
curl -fsS http://127.0.0.1:8000/packages.json |
  jq -r '["." | select(.status == "active" and .downloads >= 100)]
        | sort_by(-.downloads, .name)[]
        | "\(.name) \(.version) \(.downloads)"]'

```

结果说明：如图 2.13 所示，终端里回显出的正是 packages.json 的原始内容；curl 的三个选项各有用意：-f 让服务器返回 4xx/5xx 时以非零码退出而不是把错误页当成数据继续处理，-s 关闭进度与统计信息，-S 保证出错时仍然打印错误原因。如图 2.14 所示，jq 'length' 返回 5，说明这份 JSON 能被正确解析且是 5 条记录。如图 2.15 所示，过滤与排序后的输出依次为 delta(0.9.0, 450)、gamma(1.5.1, 450)、alpha(1.2.0, 120)：status 为 inactive 的 beta 与 downloads 只有 80 的 epsilon 被筛掉；delta 与 gamma 下载量相同，按名称字母序 delta 在前。

```

[liuchenxi@liuchenxideMacBook-Air q15 % jobs
[1] + running python3 -m http.server 8000 > server.log 2>&1
[liuchenxi@liuchenxideMacBook-Air q15 % curl -fsS http://127.0.0.1:8000/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]

```

图 2.13: 用 curl -fsS 从本地服务取回 JSON

```

[liuchenxi@liuchenxideMacBook-Air q15 % jq 'length' packages.json
5
liuchenxi@liuchenxideMacBook-Air q15 % █

```

图 2.14: jq 'length' 统计记录条数

```
liuchenxi@liuchenxideMacBook-Air q15 % curl -fsS http://127.0.0.1:8000/packages.json | jq '[.] | select(.status == "active" and .downloads >= 100)] | sort_by(-.downloads, .name)'
[
  {
    "name": "delta",
    "status": "active",
    "downloads": 450,
    "version": "0.9.0"
  },
  {
    "name": "gamma",
    "status": "active",
    "downloads": 450,
    "version": "1.5.1"
  },
  {
    "name": "alpha",
    "status": "active",
    "downloads": 120,
    "version": "1.2.0"
  }
]
```

图 2.15: jq 过滤 active 且下载量不少于 100 的记录并排序

2.4 打包交付与产物校验

题 10: 在虚拟环境中构建 wheel 并校验产物摘要

要求: 在项目的虚拟环境里把源码构建成 wheel, 查看 dist 目录, 并对产物计算 SHA-256 摘要。

操作命令 (Makefile 中 build 目标的配方就是 `python3 -m build --wheel`):

```
cd w4/q16
source .venv/bin/activate
make build
ls dist
shasum -a 256 dist/*.whl
```

结果说明: 如图 2.16 所示, 提示符前缀出现 (.venv), 说明当前 shell 用的是项目虚拟环境里的解释器与构建工具; 如图 2.17 所示, 构建最终输出 Successfully built greetlab_25020007076-0.1.0- py3- none- any.whl, dist 目录下出现了该 wheel, shasum -a 256 算出的摘要为 c3e33ba5b23525ec6c0e38cbbb48d88c8c99f42fe14b5966b2093177595d8701。反过来看, 若当前环境里没有安装 build 工具, 同一个 make build 会以 No module named build 和 *** [build] Error 1 失败 (见第一章图 1.36), 可见构建依赖同样需要被隔离在项目环境里, 才谈得上可复现的交付。

```
[(.venv) liuchenxi@liuchenxideMacBook-Air q16 % make build
python3 -m build --wheel
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for wheel...
```

图 2.16: 在项目虚拟环境中执行 make build

```
adding 'greetlab_25020007076-0.1.0.dist-info/top_level.txt'  
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'  
removing build/bdist.macosx-26.0-arm64/wheel  
Successfully built greetlab_25020007076-0.1.0-py3-none-any.whl
```

图 2.17: 成功产出 wheel 并用 `shasum -a 256` 校验摘要

2.5 本章小结

本章的 10 个实例把第四周的内容拆成了四组可独立复现的最小操作。第一组是本地质量门禁：用 `vim` 写带断言的用例、在 `pyproject.toml` 里声明 `ruff` 与 `pytest` 的配置、用 `ruff format` 与 `ruff check` 统一风格并做静态检查、用 `pytest` 跑测试、用 `chmod +x` 让 `check.sh` 可执行并通过退出码给出结论。第二组是构建自动化：把依赖写进 `Makefile` 之后，`make` 只在输入真正变新时才重跑受影响的配方，`make clean` 则只清理生成物。第三组是本地服务与数据加工：`http.server` 放到后台并用 `jobs` 管理，`curl -fsS` 保证「出错即失败、不静默失败」，`jq` 负责过滤与排序。第四组是交付：在虚拟环境里构建 `wheel`，并用 `shasum -a 256` 把产物的身份固定下来，供别人核对。10 个实例都保留了命令与输出的截图，可以按本章命令逐条复现。

解题感悟

这周的练习和上几周相比，感觉重点变了：前几周学的是一堆单独的命令，知道 `cd`、`cp`、`chmod` 各自怎么用；本周的四道题更像是要把这些命令串成一条条流程——先从复制工程、清理缓存开始，再把检查动作固化成 `check.sh`，然后用 `Makefile` 描述产物之间的依赖，最后在虚拟环境里把 `wheel` 构建出来。真正动手之后才意识到，一条命令会敲和一条流程能稳定跑通，中间差的是不少细节。

让我印象最深的是 `Make` 那道题。以前一直以为 `make` 就是把几条命令打包起来按顺序执行，直到看见第二次运行它只回了一句 `Nothing to be done for `all'.`，而 `touch data.csv` 之后它又只重跑了受影响的两步，才明白它判断的其实是「依赖文件是不是比目标新」，而不是「我是不是刚敲过这条命令」。把依赖关系写完整之后，工具就能替我做增量判断；反过来，如果依赖漏写了，它就会在需要重跑的时候偷懒，这种错误还特别不容易被发现。

前几周配好的虚拟环境，这周才算真正派上用场。`make build` 第一次执行时直接报 `No module named build`，当时有点懵，后来在 `(.venv)` 环境里再跑一次就顺利产出了 `wheel`。这件事让我理解了第一周讲的「依赖必须隔离」并不是形式要求：构建工具也是依赖，它装在哪，决定了这条构建命令在别人的机器上还能不能跑得起来。同样让我有收获的是 `check.sh` 里的 `set -euo pipefail` 和最后那个 `echo $?`——门禁的价值在于把「通过还是没通过」压缩成一个退出码，而不是靠人盯着满屏输出自己判断。

具体操作上，这周也踩了一些小坑。`jq` 的表达式一开始完全靠试，写过滤条件时把 `select` 和管道的位置搞混过好几次，最后是照着讲义里「先展开、再过滤、最后排序」的顺序一步步写才顺下来；`vim` 里改文件时也还不太熟练，经常要先 `cat` 一遍确认真的改对了。好在这周的截图我是边做边存的，写报告时不用担心漏掉某一步的操作记录，这一点比之前从容很多。

写报告本身也顺利了一些。上周刚学会用 `LaTeX` 写东西的时候，光是调格式就花了不少时间；这次图片、代码块和交叉引用基本都是照搬上次的写法，用 `\label` 和 `\ref` 引用图片也比手动对着编号数要省心，所以能把更多时间花在把每一步的实验结果说清楚上。当然不足还是很明显的：查资料的时间依然不少，尤其是 `jq` 和 `Make` 的语法细节；有些快捷键和环境配置也只是照做，还没到能给别人解释清楚的程度。希望接下来几周能把这些常用操作练到不用查资料，让实验的重心从「跑通命令」慢慢转到「想清楚流程」上。

